

SYNOPSIS
ON
“AURIX: An Agentic Unified Risk Intelligence Platform with
Autonomous Detection and Remediation”

Submitted in
Partial Fulfillment of requirements for the Award of Degree
of
Bachelor of Technology
In
Computer Science and Engineering
(Artificial Intelligence)
By

(Project Id: 27_CSAI_4A_09)

Bhavya Gupta (2301641520064)
Bhumika Mishra (2301641520065)
Divyansh Gaur (2301641520070)
Divyanshi Singh (2301641520072)

Under the supervision of
Dr. Jyoti Dua
(Associate Professor)



Pranveer Singh Institute of Technology.

Kanpur - Agra - Delhi National Highway - 19
Bhauti - Kanpur - 209305.

(Dr. A.P.J. Abdul Kalam Technical University)

1. Introduction

Modern software development, accelerated by AI-driven coding tools, allows engineering teams to ship code at an unprecedented pace. However, this high velocity creates a massive "security gap." While code is deployed daily, deep security validation and penetration testing typically occur infrequently. Consequently, organizations unknowingly ship vulnerabilities to production, exposing them to critical cyber threats.

To address this, we introduce **AURIX: An Agentic Unified Risk Intelligence Platform**. AURIX functions as an autonomous, self-healing ecosystem that provides on-demand security validation and remediation for every code build. By integrating intelligent scanning with autonomous patching, it ensures that security defenses move at the exact same speed as software innovation.

Key Technical Terms:

- **SAST & SCA:** Static Application Security Testing (analyzing source code) and Software Composition Analysis (analyzing third-party dependencies).
- **Agentic Wargaming:** A dual-AI simulation where a "Red Agent" acts as an attacker to exploit a vulnerability, and a "Blue Agent" acts as a defender to create a patch.
- **LangGraph Orchestration:** A stateful, multi-actor AI framework used to manage and route the complex reasoning loops between the Red and Blue agents.
- **Retrieval-Augmented Generation (RAG):** An AI framework used to ground the agents' reasoning in up-to-date threat intelligence (e.g., CVE databases) for accurate, context-aware analysis.
- **Ephemeral Sandbox (Docker/gVisor):** A highly secure, temporary, and isolated container environment used to safely execute malicious Proof-of-Concept (PoC) scripts without harming the host system.

2. Project Objective

The primary objective of this project is to build an autonomous system that not only

detects software vulnerabilities but conclusively proves their exploitability and automatically writes verified patches.

Specifically, the project aims to achieve the following:

- **Unified Intelligence Scanning:** Analyze source code, third-party dependencies, and infrastructure configurations simultaneously to identify complex, cross-layer risks.
- **Eliminate Alert Fatigue:** Use AI agents to write actual exploit scripts (PoCs), transitioning security from theoretical "maybe" alerts to undeniable proof. If a vulnerability cannot be exploited in the sandbox, it is filtered out.
- **Automate Remediation (Self-Healing):** Autonomously generate code patches, verify them against the PoC in a secure environment, and open a Pull Request (PR) directly on GitHub.

End-User Benefits:

- Ultimately, AURIX is designed to significantly reduce the cognitive and manual burden on developers and security teams. By delivering verified patches and actionable insights through an intuitive web dashboard (featuring visual risk heatmaps and triage boards) and a native VS Code extension, it empowers developers to maintain a secure codebase without needing specialized security expertise, allowing them to focus entirely on software innovation..

3. Literature Survey

The foundation of AURIX is built upon an analysis of existing enterprise security solutions and recent academic advancements in Automated Program Repair (APR). The current software security landscape can be categorized as follows:

- **Traditional SAST and SCA Tools:** Foundational scanning tools such as SonarQube, PMD, and Semgrep are utilized for Static Application Security Testing (SAST), while tools like Trivy are used for Software Composition Analysis (SCA). While highly effective at baseline detection, they operate in

silos and lack the context to definitively prove exploitability.

- **Commercial Enterprise Solutions:** Industry leaders like Snyk, Wiz, and Checkmarx often require multiple separate tools to scan code, libraries, and secrets. They rely on static, rule-based logic that frequently leads to "False Positives" and alert fatigue. Furthermore, these platforms typically provide theoretical risk scores and "suggested" code fixes that may break the application or fail to work in a live environment.
- **The AURIX Orchestration Approach:** Rather than competing with foundational scanners, AURIX integrates tools like Semgrep, Trivy, Gitleaks, and Hadolint into a unified multi-layer engine. This allows the platform to normalize fragmented tool outputs into a single intelligence schema to identify cross-layer risks.
- **Agentic AI and Autonomous Repair:** Recent research demonstrates that leveraging Large Language Models (LLMs) through multi-agent workflows can simulate complex human reasoning. AURIX utilizes an autonomous "Red Agent" to generate an executable Proof-of-Concept (PoC) and a "Blue Agent" to create a patch. These agents simulate real attacks to prove exploitability, and patches are verified in a secure gVisor sandbox.

The AURIX Advantage:

AURIX overcomes the limitations of existing tools through four streamlined advantages:

- **Contextual Awareness:** Identifies cross-layer risks completely missed by siloed scanning tools.
- **Zero Noise:** Eliminates alert fatigue by delivering executable PoC scripts instead of theoretical alerts.
- **Guaranteed Fixes:** Uses closed-loop sandbox verification to ensure generated patches work without breaking the build.
- **End-to-End Automation:** Autonomously identifies visual "Hot Zones" and generates GitHub Pull Requests with zero manual developer intervention.

4. Feasibility Study:

Technical Feasibility: The proposed architecture relies on established and robust modern frameworks. The Backend Gateway leverages FastAPI/Node.js, with PostgreSQL and Redis (via Upstash) handling state management and task queuing. The AI capabilities are powered by advanced Large Language Models (LLMs) orchestrated via LangGraph, which allows for stateful, cyclical reasoning loops (Reflexion). Safe execution of malicious code is highly feasible using Docker containerisation combined with strict network isolation and gVisor sandboxing.

Operational Feasibility: AURIX is designed to seamlessly integrate into existing developer workflows rather than disrupt them. By offering both a VS Code Extension (for local "Delta" scans and pre-flight secret guards) and a Web Dashboard (for cloud-based GitHub ingestion), it caters to various operational preferences. The "One-Click PR" feature ensures that fixing bugs requires minimal manual intervention, making it highly feasible for fast-paced agile teams.

Economic Feasibility: The underlying scanning engines (Semgrep, Trivy, Gitleaks, Hadolint) are open-source, eliminating heavy licensing costs for the core detection logic. The primary cost driver will be LLM API usage and cloud compute. However, by implementing a Redis-backed rate limiter, an automated data sanitiser, and a strict "Consensus Gate" (filtering low-confidence bugs before they reach the expensive AI agents), the system optimises compute resources effectively.

5. Methodology/ Planning of work

The development of AURIX is divided into a robust, multi-layered architecture, developed concurrently across integrated domains to ensure scalability, accurate threat detection, and secure autonomous remediation.

Step 1: Client Interfaces & Data Ingestion (Layer 1)

- **VS Code Integration:** A local IDE extension safely packages user code into a .zip payload, strictly adhering to .gitignore rules to prevent node_modules bloat. It features a "Pre-Flight Secret Guard" to prevent the accidental upload of

hardcoded API keys.

- **Web Dashboard:** A Next.js/React application providing a dual-mode ingestion UI (URL pasting or native GitHub OAuth syncing). It utilizes real-time asynchronous polling to update the user as the AI processes their code.

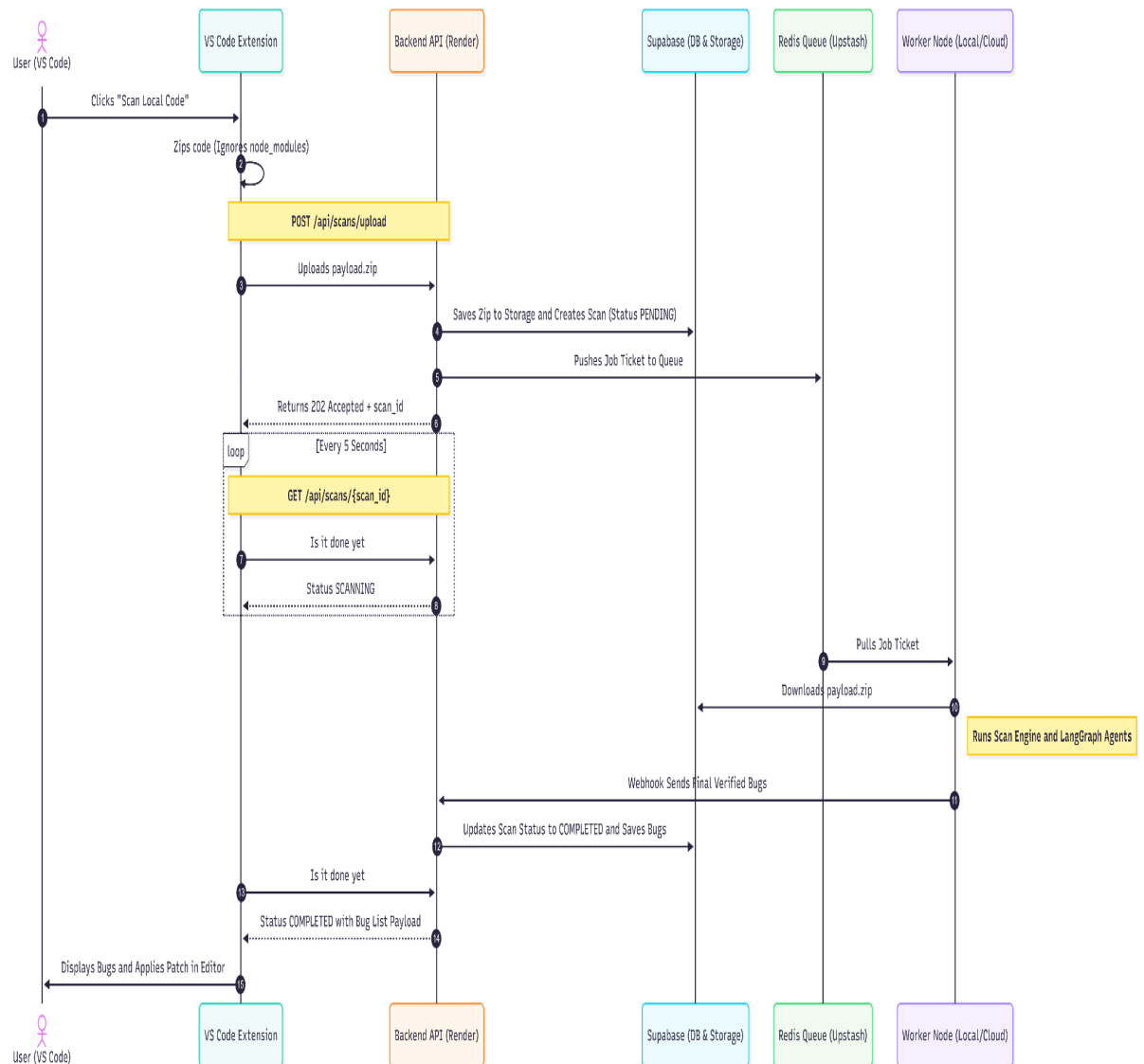


Fig.1. VS Code Extension Sequence Diagram

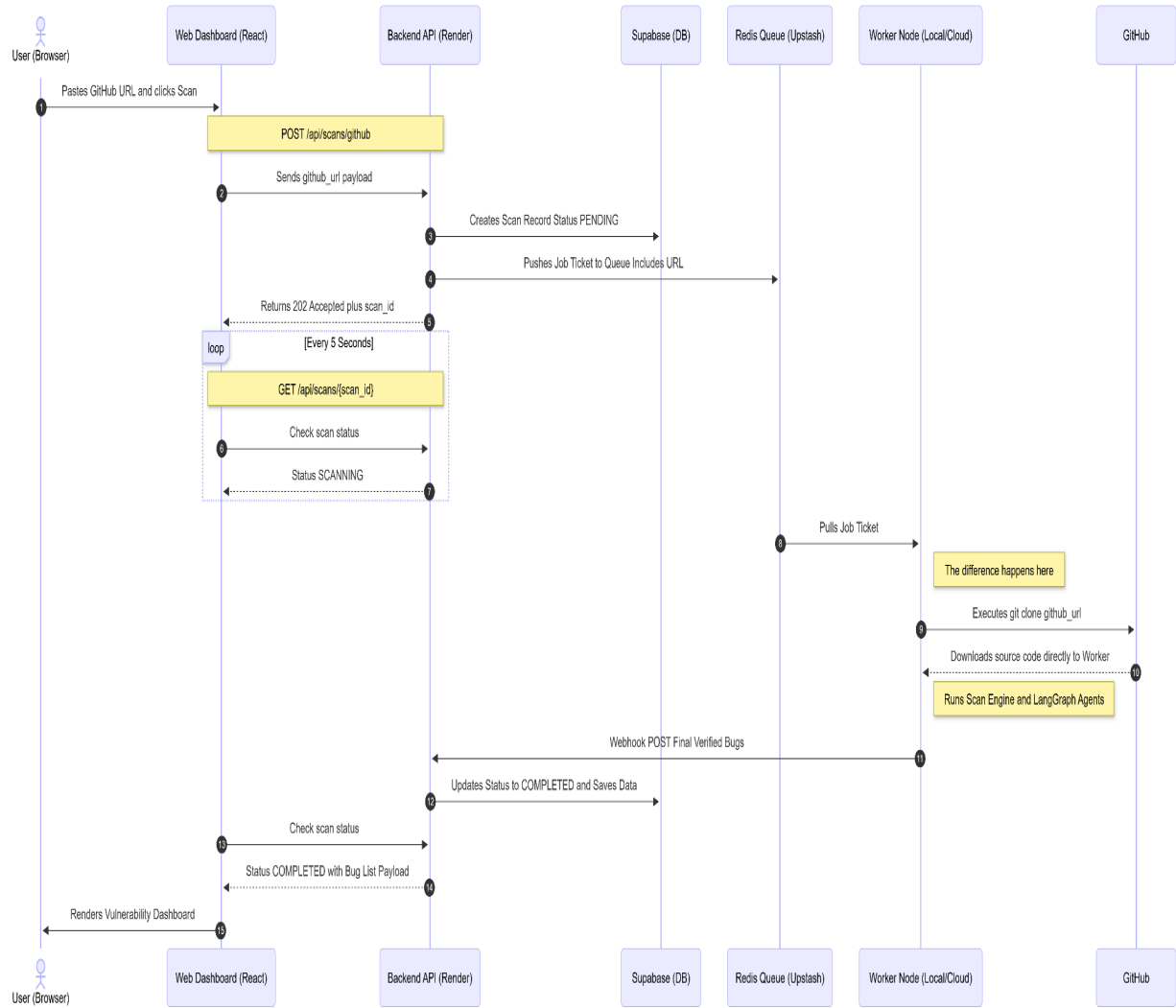


Fig.2. Web Dashboard Sequence Diagram

Step 2: Backend API & Storage Gateway (Layer 2)

- The Hub & Vault:** A centralized REST API (FastAPI/Node.js) authenticates users via JWTs. It safely receives codebase payloads, uploads them to Supabase Cloud Storage, logs the scan metadata into a PostgreSQL database, and pushes the job ticket to a Redis Task Queue (Upstash) for asynchronous processing.

USERS			
uuid	id	PK	Managed by Supabase Auth
string	email		
timestamp	created_at		


 creates & owns

PROJECTS			
uuid	id	PK	
uuid	user_id	FK	Links project to specific user
string	name		e.g., NodeJS Goof App
string	repository_url		GitHub Link (NULL if local scan)
timestamp	created_at		


 has many

SCANS			
uuid	id	PK	
uuid	project_id	FK	
string	status		PENDING, SCANNING, COMPLETED, FAILED
string	storage_path		Supabase Storage URL for local .zip uploads
int	total_findings		Raw count from Opengrep/Trivy
int	verified_findings		Count after AI filtering
timestamp	started_at		
timestamp	completed_at		



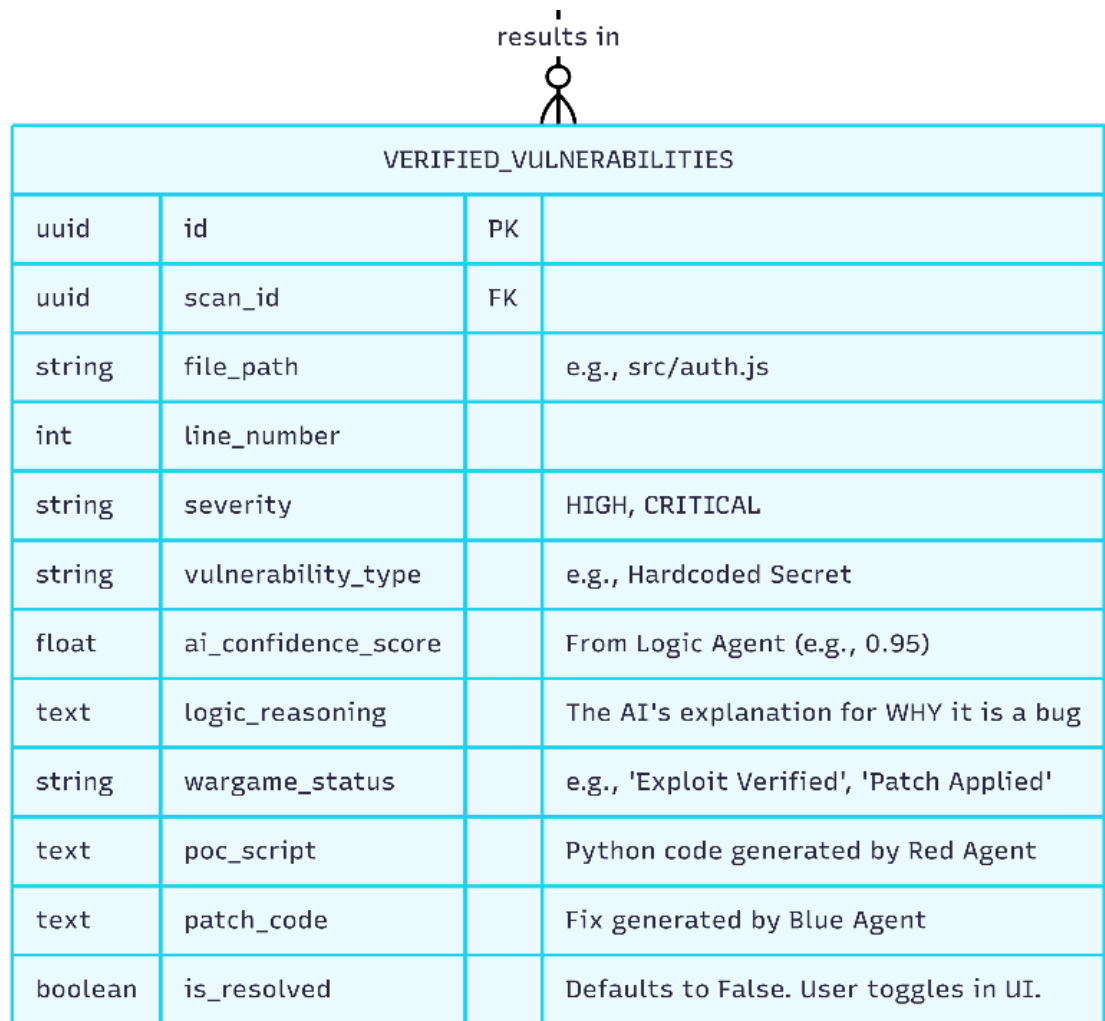


Fig.3. Database ER Diagram

Step 3: AI Orchestration & Logic Processing (Layer 3 & 4)

- **Triage & Context Slicer:** A dedicated worker node pulls the task from Redis, unzips the payload, and runs parallel static baseline scans (using Semgrep, Trivy, Gitleaks, and Hadolint). A "Semantic Slicer" then extracts relevant multi-file execution paths from these raw alerts.
- **LangGraph State Machine:** The sliced context is passed to the AI Logic Agent via LangGraph. If the agent's confidence score regarding the exploitability of the flaw is ≥ 0.80 , the system routes the data to the dual-agent wargaming loop.

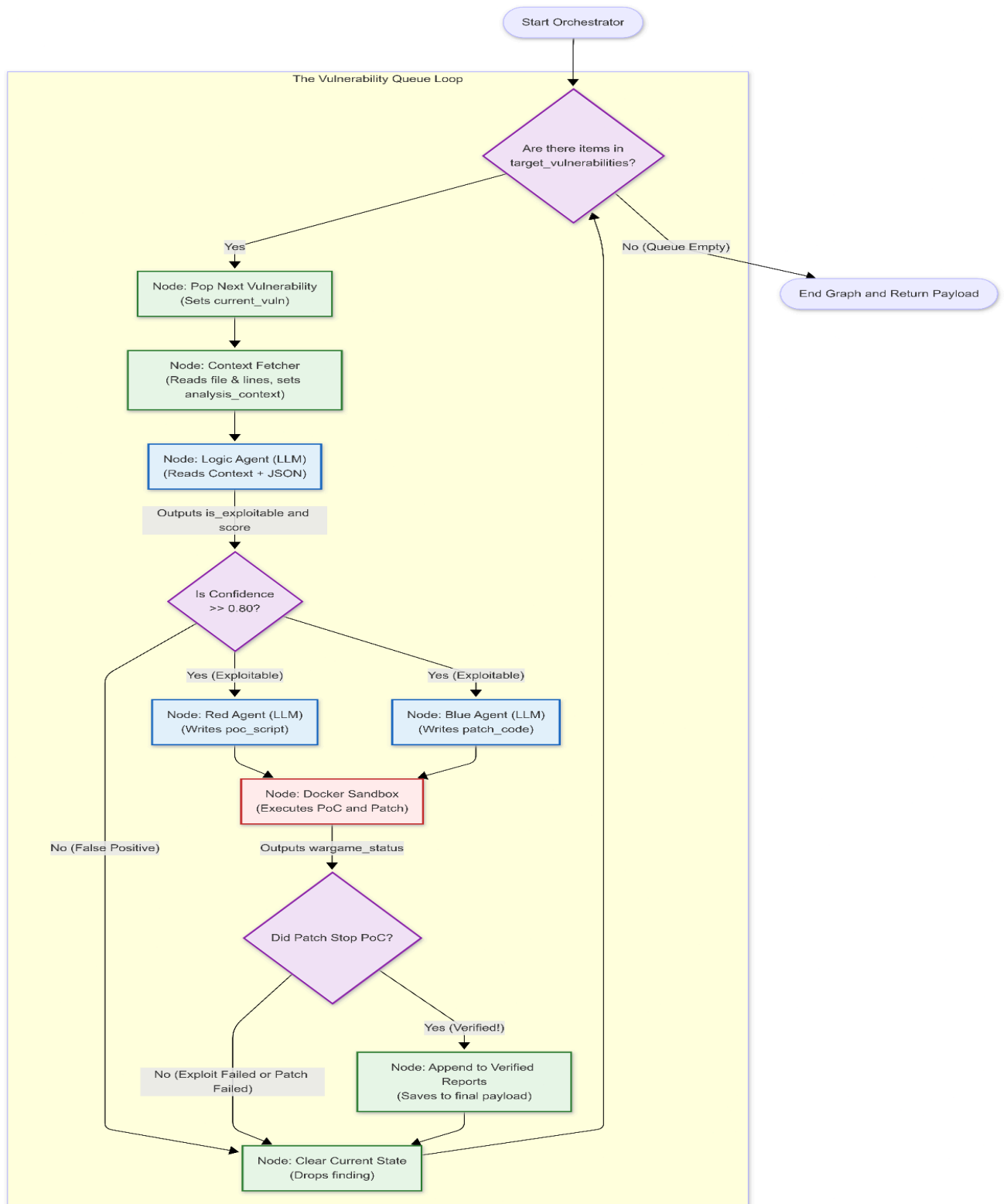
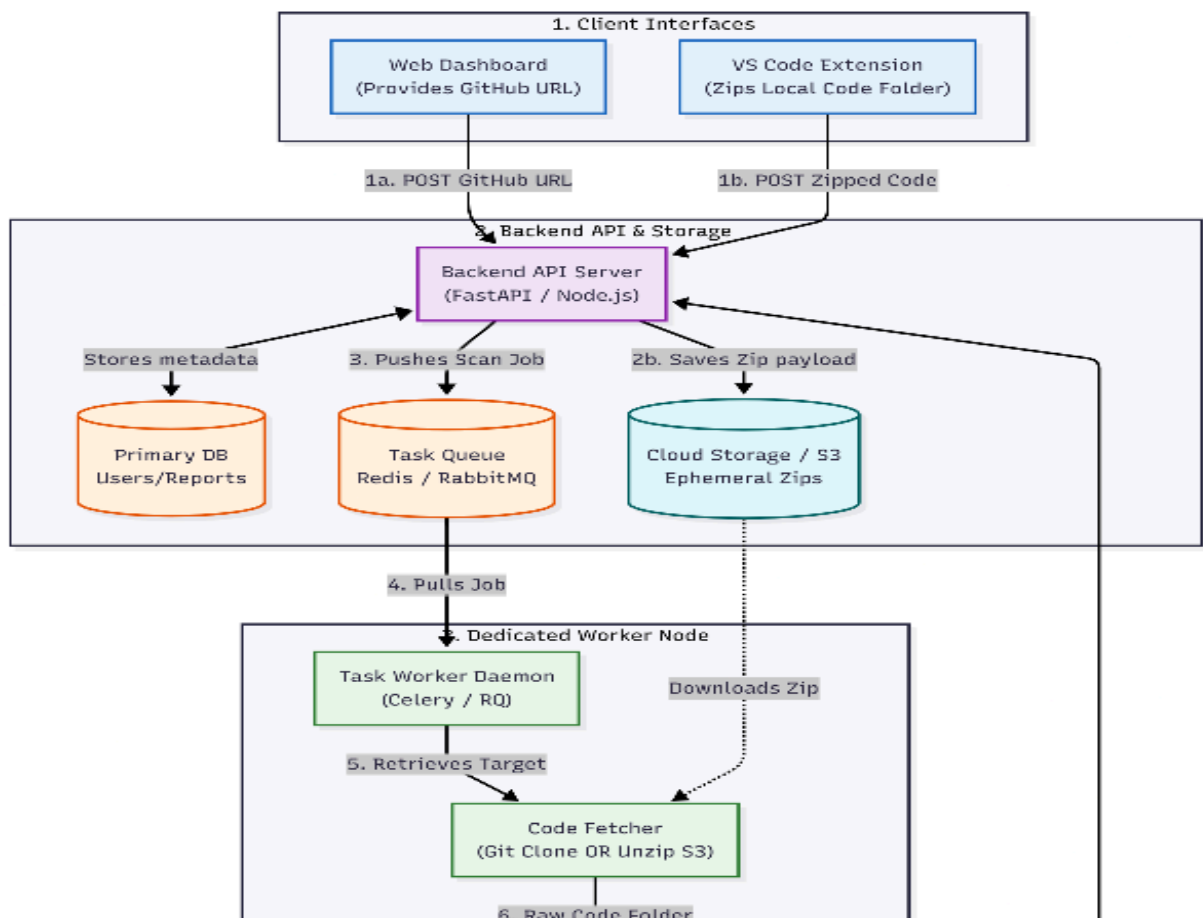
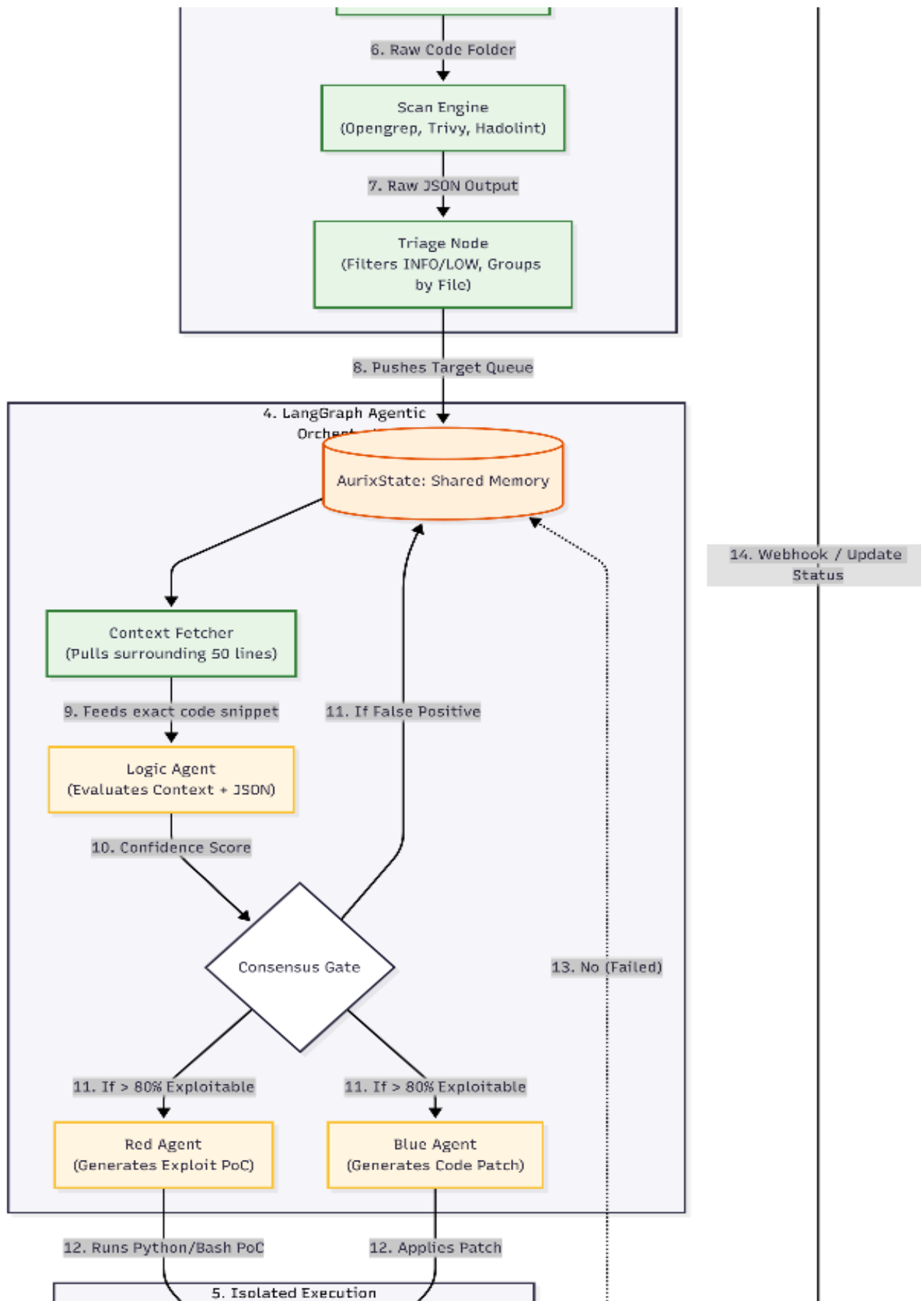


Fig.4.LangGraph State Machine Orchestrator (LLD) Diagram

Step 4: Isolated Wargaming & Self-Healing (Layer 5)

- **Red vs. Blue Agents:** The Red Agent autonomously writes a Python Proof-of-Concept (`poc_script.py`), while the Blue Agent generates a replacement code patch.
- **Ephemeral Sandbox:** A Docker container with strict network isolation spins up. The PoC is executed. If it fails due to syntax errors, a "Reflexion Loop" sends the logs back to the Red Agent for self-correction. If the PoC successfully exploits the mock environment, the Blue Agent's patch is applied, and the exploit is run again to verify the fix.
- **Final Output:** Verified vulnerabilities are passed back to the Web Dashboard's Kanban board, where developers can review the visual risk heatmap and apply the verified patch directly to their repository via an automated GitHub Pull Request.





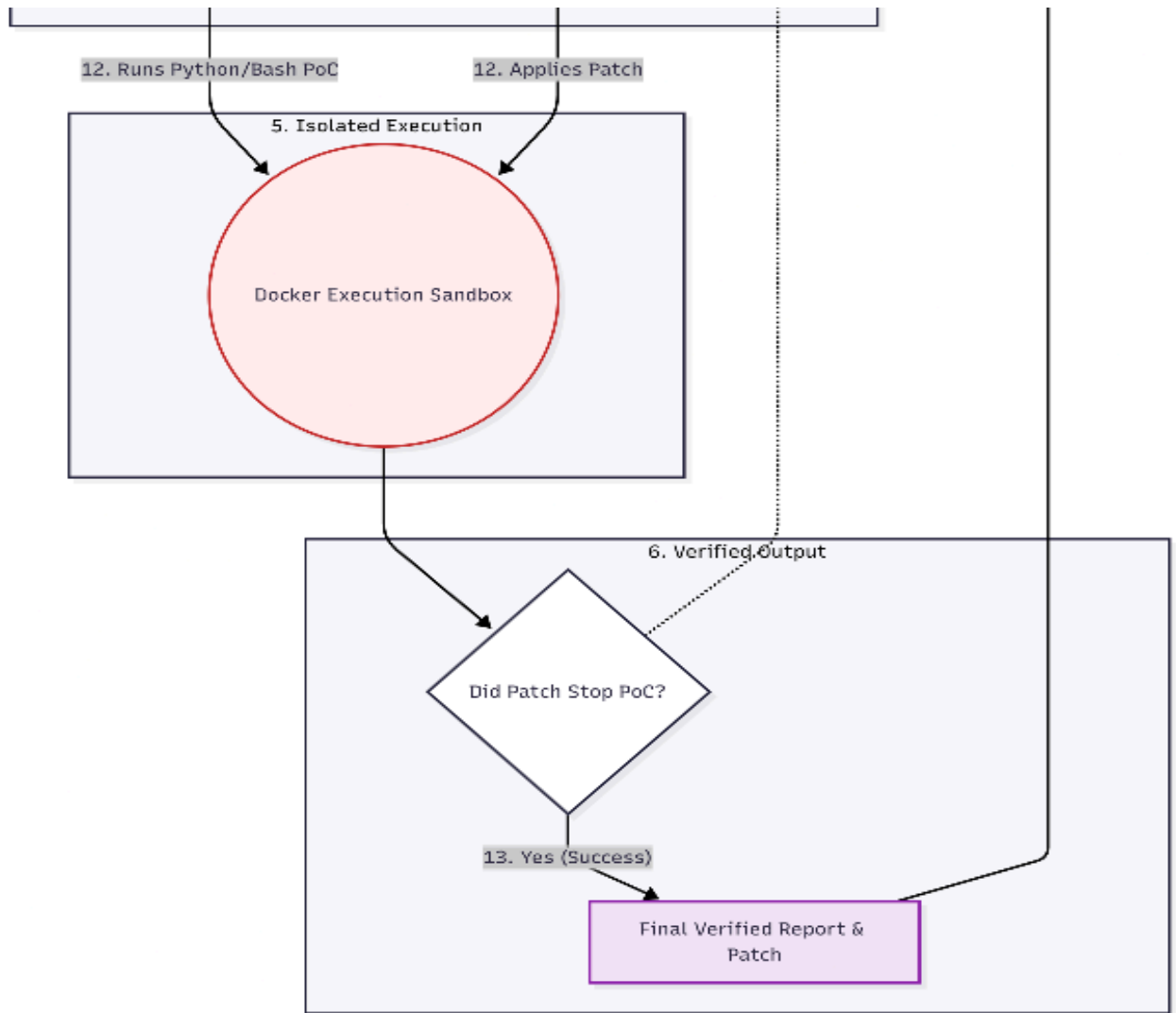


Fig.5. AURIX High-Level Design (HLD) Diagram

6. Tools/Technology Used:

6.1 Hardware Requirements

- Processor: Intel Core i5/i7 10th Gen or higher / AMD Ryzen 5/7 5000 series.
- RAM: Minimum 16 GB (essential for running Docker sandboxes and local IDE extensions simultaneously).
- Storage: Minimum 256 GB SSD.
- Network: Stable internet connection for repository access, dependency scanning, and API communication.

6.2 Software Requirements

- Operating System: Linux (Ubuntu 22.04 LTS) or Windows with WSL2.
- Development Tools: Visual Studio Code, Python 3.11+, Node.js, Docker, gVisor, Git.
- Database & Queues: PostgreSQL (via Supabase), Redis (via Upstash).
- Security Scanners: Semgrep (SAST), Trivy (SCA), Gitleaks (Secrets), Hadolint (Infrastructure).
- Frontend Technologies: React.js / Next.js, Tailwind CSS.
- AI & APIs: LangGraph for orchestration, Google Gemini Pro / OpenAI GPT-4o APIs, GitHub REST API.

7. References

- [1] Q. Xu, Z. Sheng, Z. Chen, and J. Huang, “A Systematic study of LLM-Based architectures for Automated Patching,” *arXiv.org*, Mar. 01, 2026. Available: <https://arxiv.org/abs/2603.01257>
- [2] D. Toprani and V. K. Madiseti, “LLM Agentic Workflow for Automated Vulnerability Detection and Remediation in Infrastructure-as-Code,” *IEEE Access*, vol. 13, pp. 69175–69181, Jan. 2025, doi: 10.1109/access.2025.3560911. Available: https://www.researchgate.net/publication/390823751_LLM_Agentic_Workflow_for_Automated_Vulnerability_Detection_and_Remediation_in_Infrastructure-as-Code
- [3] “LLM4CVE: Enabling Iterative Automated Vulnerability Repair with Large Language Models.” Available: <https://arxiv.org/html/2501.03446v1#S6>
- [4] H. Xu *et al.*, “RedAgent: Red Teaming Large Language Models with Context-aware Autonomous Language Agent,” *arXiv.org*, Jul. 23, 2024. Available: <https://arxiv.org/abs/2407.16667>
- [5] V. Saravanan, “Beyond CVES: Agentic AI for Real-World Software vulnerability discovery and Prioritization,” *International Journal of Artificial Intelligence Data Science and Machine Learning*, vol. 7, no. 1, pp. 130–134, Jan. 2026, doi: 10.63282/3050-9262.ijaidsm-l-v7i1p124. Available: <http://ijaidsm-l.org/index.php/ijaidsm-l/article/view/422>